accomplished with the `{$sapi}` portion. Don't worry if you're not sure about any of this just yet: we'll cover variables in Chapter 2, "Data, Types, and Variables," and the many ways to handle text in the appropriately named Chapter 8, "Strings."

If you request `index2.php` in the same way that `index.php` was, you should have the following in your web browser:

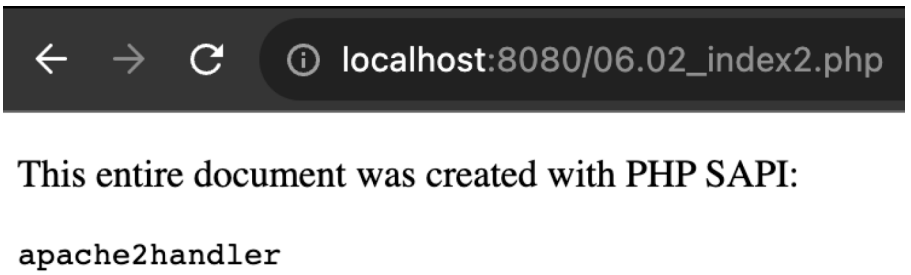


Figure 1-3. *Pure PHP*

Notice that in `index2.php`, we did not use a closing `?>` tag. Closing tags should not be used in files that contain only PHP code, as they can trigger the sending of output to the browser: in a complex, multi-file PHP application, this might cause unwanted behavior if other PHP code has not had a chance to execute yet.

Introduction to PHP Syntax

Now we come to a very important topic: PHP's syntax. Your future as a PHP programmer (or indeed a programmer in general) depends largely on your grasp of language syntax: the rules by which the language is used. No pressure! The full syntax rules of PHP (or any general-purpose language) would cover dozens of pages, and attempting to reproduce them here would be a good way for me to ensure you don't finish this chapter. That said, we can tackle some fundamentals without overloading you too much at this point. For any readers that have experience with C or C++: you find PHP's syntax familiar, and that's not a coincidence. Like other languages implemented in C, PHP borrows a lot of syntax rules from it.

We can abstract PHP's syntax rules in a top-down way like this: valid PHP is composed of a sequence of *statements* (of various types) that are themselves composed